Dr. Harwinder Singh Sohal
Associate Professor (Department of Computer Science and Engineering)

OOP-Focused Case Study Questions (C++ / Java)

□ Case Study 1: Online Learning Platform

A platform has different users: Student, Instructor, and Admin. Each user has login functionality, but their permissions differ.

Questions:

1. How will you apply **inheritance** to model different user types?
2. Which methods should be **overridden** in child classes?
3. How can **polymorphism** be used for login behavior?
4. How does **encapsulation** protect user credentials?

□ Case Study 2: Vehicle Rental System

A system manages vehicles like Car, Bike, and Truck. Each has a different rental cost calculation.

Questions:

1. Design a class structure using **inheritance and abstraction**.
2. How will you implement **runtime polymorphism** for cost calculation?
3. What is the role of an **abstract class vs interface**?
4. Explain **dynamic binding** in this scenario.

□ Case Study 3: Payment Gateway Integration

Different payment modes (Credit Card, UPI, Net Banking) follow different validation and processing logic.

Questions:

1. How would you design this using **interfaces (Java)** or **pure virtual functions (C++)**?
2. Explain **loose coupling using abstraction**.
3. How does **polymorphism** simplify adding new payment methods?
4. What is the benefit of **method overriding** here?

□ Case Study 4: Employee Management System

An organization has employees like Manager, Developer, and Intern with different salary calculations.

Questions:

1. How will **inheritance hierarchy** be structured?
2. Implement **method overriding** for salary computation.
3. Explain **base class pointer with derived class object** (C++ concept).
4. How does **polymorphism improve flexibility**?

□ Case Study 5: Shape Drawing Application

A graphics app supports shapes like Circle, Rectangle, and Triangle with a `draw()` method.

Questions:

1. How will you implement **runtime polymorphism**?
2. What is the difference between **overloading and overriding** in this case?
3. Explain **early vs late binding**.
4. Why is abstraction important here?

□ Case Study 6: Banking System Security

Sensitive data like balance and PIN should not be accessed directly.

Questions:

1. How does **encapsulation ensure data security**?
2. Implement **private variables with public methods**.
3. What are **access modifiers** and their importance?
4. What happens if encapsulation is violated?

□ Case Study 7: Smart Home Automation

Devices like Light, Fan, and AC respond to commands like ON/OFF.

Questions:

1. How would you design a **common interface** for devices?
2. Explain **polymorphism using a single command interface**.
3. What is **method overriding in derived classes**?
4. How does this design improve scalability?

□ Case Study 8: E-Commerce Product Catalog

Products like Electronics and Clothing share some properties but differ in tax calculation.

Questions:

1. Apply **inheritance and abstraction**.
2. Implement **polymorphic tax calculation**.
3. What is **code reusability**, and how is it achieved?
4. How does OOP reduce redundancy?

□ Case Study 9: Game Development

A game has characters like Warrior, Mage, and Archer with different attack behaviors.

Questions:

1. How will you use **method overriding** for attack()?
2. Explain **runtime polymorphism with base class reference**.
3. What is **dynamic binding**?
4. How can you extend the system for new characters?

□ Case Study 10: Library Management System

Books, Journals, and Magazines have different borrowing rules.

Questions:

1. Design using **abstract classes**.

2. Implement **different borrowing policies using overriding**.
3. Explain **hierarchical inheritance**.
4. What are advantages of OOP in such systems?

Advanced OOP Case Studies (Interview Level)

□ Case Study 11: Diamond Problem (Multiple Inheritance)

A class inherits from two classes that have a common base class.

Questions:

1. What is the **diamond problem**?
2. How is it resolved in **C++ (virtual inheritance)**?
3. Why does Java avoid this issue?
4. What is the role of **interfaces in Java**?

□ Case Study 12: Object Slicing (C++)

Assigning a derived class object to a base class object leads to data loss.

Questions:

1. What is **object slicing**?
2. Why does it occur?
3. How can it be prevented using **pointers/references**?
4. Explain real-world impact.

□ Case Study 13: Immutable Class Design (Java)

A class should not allow modification after object creation.

Questions:

1. How do you create an **immutable class**?
2. Why is **encapsulation critical** here?
3. What are benefits in multithreading?

4. Give examples like String class.

□ Case Study 14: Constructor and Destructor Flow

Objects are created and destroyed dynamically.

Questions:

1. What is the **order of constructor execution in inheritance**?
2. Explain **destructor order** in C++.
3. Difference between **constructor overloading** and default constructor.
4. Why are destructors important?

□ Case Study 15: Loose vs Tight Coupling

A system is hard to maintain due to tightly coupled classes.

Questions:

1. What is **tight coupling**?
2. How does **abstraction reduce coupling**?
3. Use **interfaces to redesign the system**.
4. Benefits of loose coupling in real-world applications.